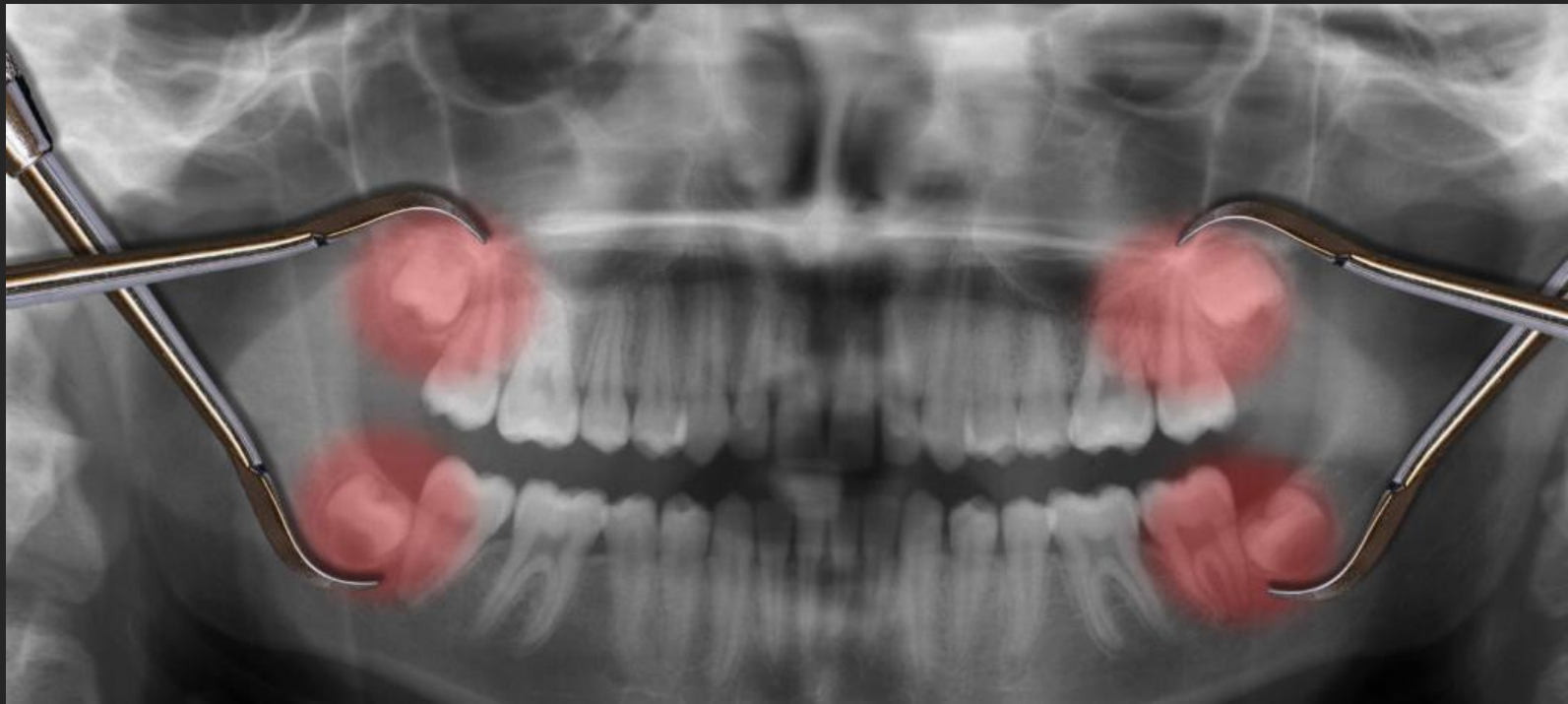


Final Project - Dental Image Processing

BY:

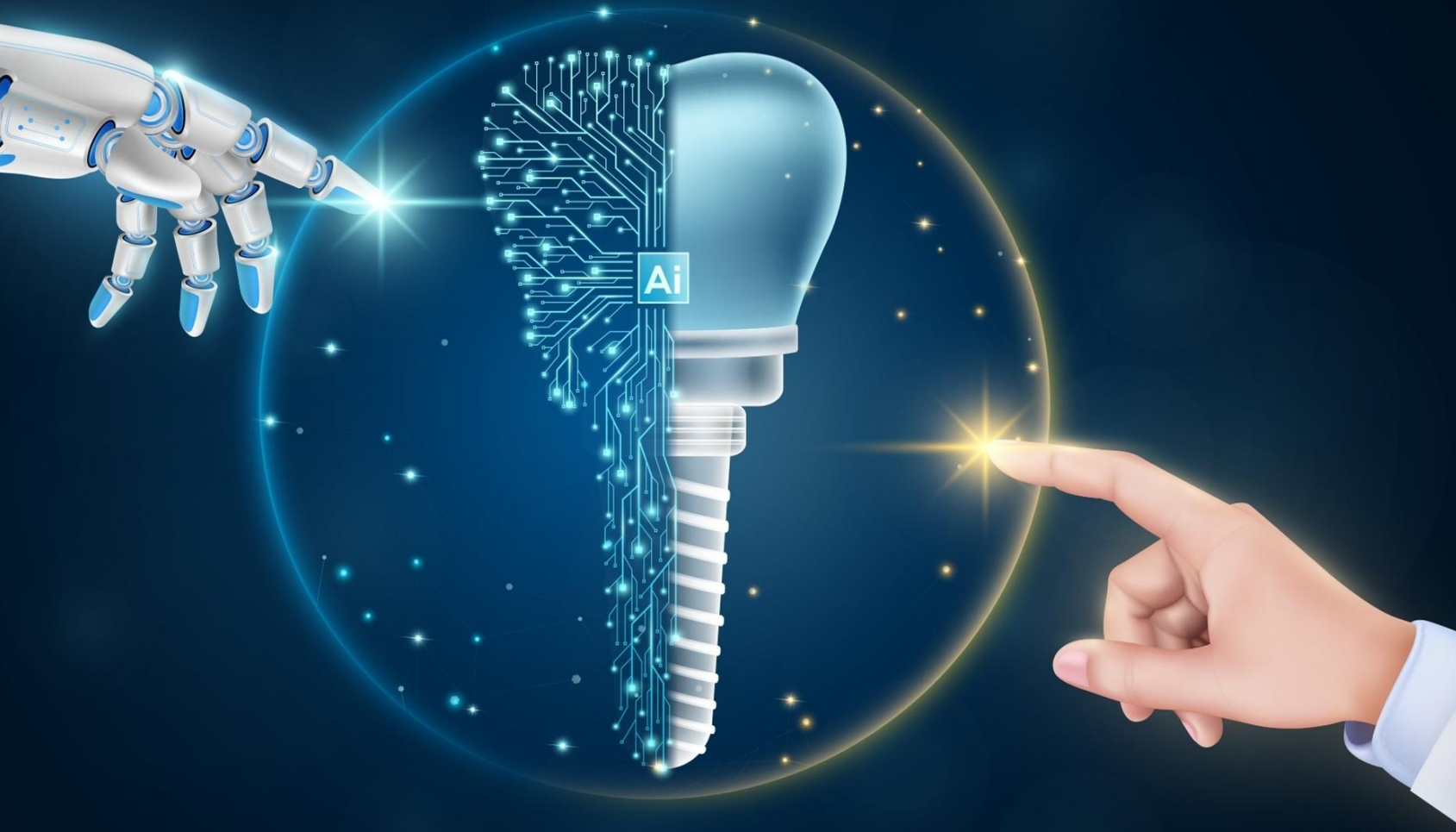
- KETAN KHANNA
- VIJAY CHARLES
- NIYOOSHA SALIMI



Convolutional Neural Networks (CNN) with Image Processing to Train Dental Image Model

Objective:

- This presentation explores how Convolutional Neural Networks (CNNs) are employed to automate dental image analysis with respect to detecting dental conditions such as cavities, tooth fractures, periodontal disease and other abnormalities.
- By leveraging CNNs along with image processing techniques, this project aims to build a high-accuracy diagnostic model that can assist dental professionals, expedite diagnosis, and enhance patient outcome.



Data Cleaning

Data Cleaning Process:

- Handle Missing Values
- Remove Duplicates
- Handle Outliers
- Correct Data Types
- Standardize Column Names
- Feature Engineering
- Encoding & Normalization
- Save Cleaned Data

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
Processing file: /content/drive/My Drive/final project/train/train.csv
Handling missing values...
Removing duplicates...
Handling outliers...
Standardizing column names...
Encoding categorical columns...
Normalizing numerical columns...
```

Data Cleaning Summary:

Shape of data after cleaning: (8030, 8)

Missing values after cleaning: filename 0

width 0
height 0
class 0
xmin 0
ymin 0
xmax 0
ymax 0

dtype: int64

First few rows of the cleaned data:

	filename	width	height	class	xmin	ymin	xmax	ymax
0	760	0.0	0.0	3	-0.847740	0.325326	-0.765048	0.076459
1	760	0.0	0.0	1	-0.908251	0.106697	-0.970743	-0.546824
2	760	0.0	0.0	3	-0.291043	0.575188	-0.147964	0.855562
3	760	0.0	0.0	3	0.689228	0.637654	0.723213	0.762069
4	760	0.0	0.0	3	1.088598	0.262861	1.098304	0.107623

Cleaned data saved to: /content/drive/My Drive/final project/train/train_cleaned.csv

File not found: /content/drive/My Drive/final project/validate/validat.csv

```
Processing file: /content/drive/My Drive/final project/test/test.csv
Handling missing values...
Removing duplicates...
Handling outliers...
Standardizing column names...
Encoding categorical columns...
Normalizing numerical columns...
```

Data Cleaning Summary:

Shape of data after cleaning: (473, 8)

Missing values after cleaning: filename 0

width 0
height 0
class 0
xmin 0
ymin 0
xmax 0
ymax 0

dtype: int64

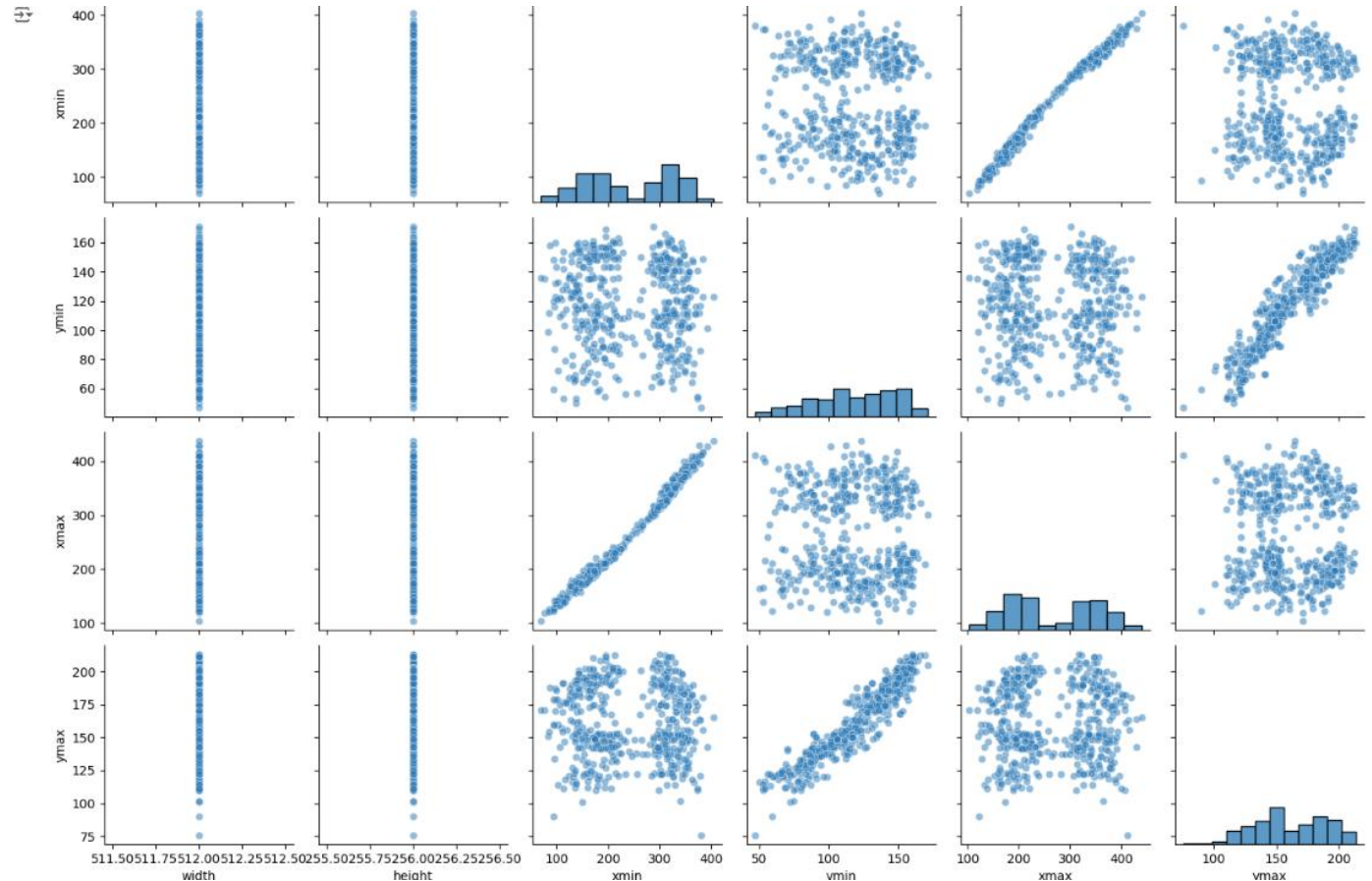
First few rows of the cleaned data:

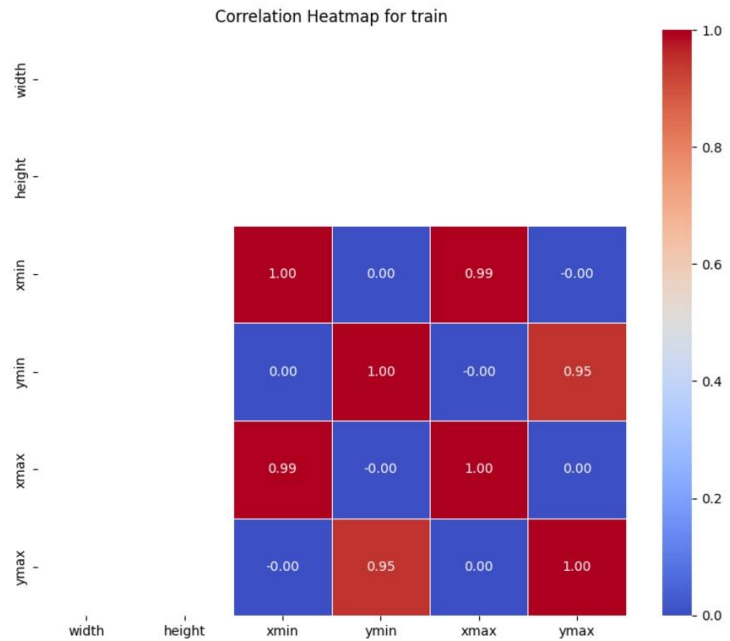
	filename	width	height	class	xmin	ymin	xmax	ymax
0	12	0.0	0.0	3	-0.543239	1.756673	-0.669525	1.829073
1	12	0.0	0.0	3	0.545194	1.825680	0.411994	1.577293
2	12	0.0	0.0	3	-0.449611	-0.382526	-0.575480	-0.400977
3	12	0.0	0.0	3	0.533490	-0.486036	0.423750	-0.400977
4	12	0.0	0.0	3	0.697341	-0.520539	0.541306	-0.400977

Cleaned data saved to: /content/drive/My Drive/final project/test/test_cleaned.csv

Data Exploratory

- Display basic dataset information using `df.info()`
- Statistical Summary o Summary of numerical columns using `df.describe()`
- Missing Values Check o Identify missing values with `df.isnull().sum()`
- Visualizations
 - Histograms: Distribution of numerical features
 - Correlation Heatmap: Visualize correlations between numerical features o
 - Count Plots: Visualize distributions of categorical variables
- Pairplot: Check relationships between numerica





Dataset Features: Correlation Heatmap

A correlation heatmap is a graphical representation of the relationships between numerical variables in a dataset.

It highlights:

- 1) Positive correlation (+1): Variables move together.
- 2) Negative correlation (-1): Variables move oppositely.
- 3) No correlation (0): No linear relationship.

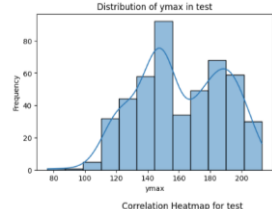
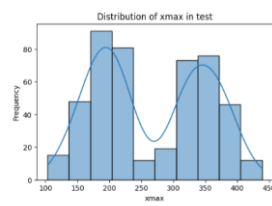
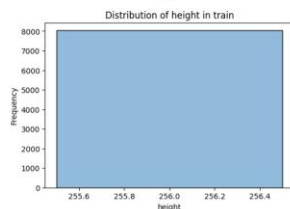
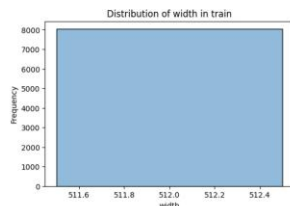


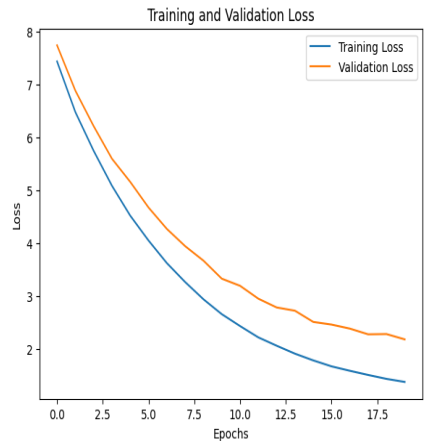
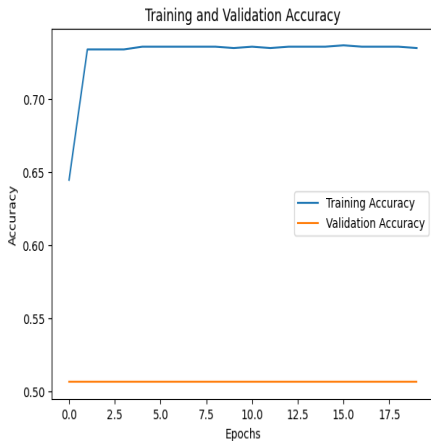
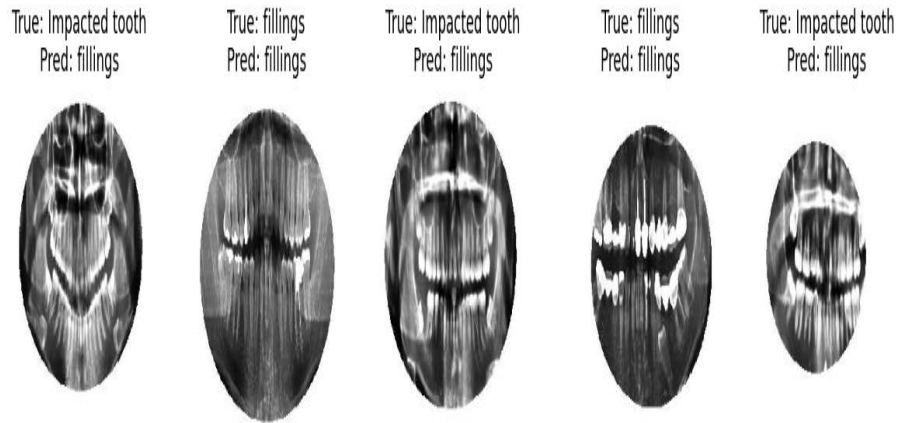
Image Classification Model (4 Categories)

- **Model Architecture**
 - **Conv2D Layers:** 3 convolutional layers with increasing filters (32, 64, 128)
 - **MaxPooling2D Layers:** Reducing spatial dimensions after each Conv2D layer
 - **Flatten:** Flattening the output to feed into fully connected layers
 - **Dense Layers:** 128 neurons with ReLU activation for learning complex features
 - **Dropout:** Regularization (50% dropout rate) to prevent overfitting
 - **Output Layer:** Softmax activation for 4 classes
- **Compilation**
 - **Optimizer:** Adam
 - **Loss Function:** Categorical crossentropy (for multi-class classification)
 - **Metrics:** Accuracy

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)  
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_4 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_5 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense_2 (Dense)	(None, 128)	3,211,392
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 4)	516

Total params: 3,305,156 (12.61 MB)
Trainable params: 3,305,156 (12.61 MB)
Non-trainable params: 0 (0.00 B)



VGG16 Model for Image Classification

Model Architecture

- **Pre-trained VGG16:** Using ImageNet weights, excluding top layers
- **Custom Layers:**
 - Global Average Pooling
 - Dense Layer with L2 Regularization
 - Dropout (50%) to prevent overfitting
 - Softmax activation for multi-class output (4 classes)

Image Preprocessing

- **Data Augmentation:** Random rotations, shifts, zooms, flips
- **Rescaling:** Normalization to [0, 1] for images

Model Training

- **Optimizer:** Adam with learning rate 0.0001
- **Loss Function:** Categorical Crossentropy
- **Callbacks:** EarlyStopping and ModelCheckpoint

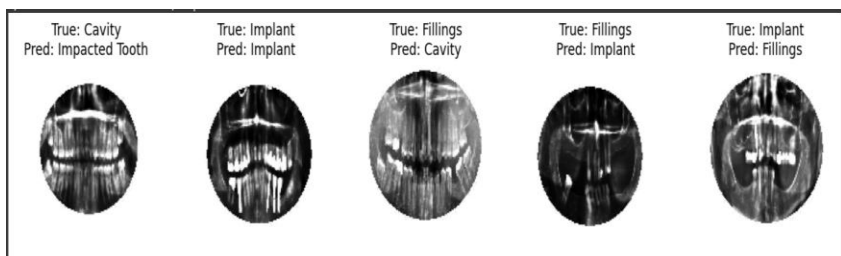
Visualization

- Predictions vs True Labels
- Training/Validation Accuracy & Loss Plots

```

Training Improved Model...
Epoch 1/20
61/61 736s 11s/step - accuracy: 0.3532 - loss: 2.7234 - val_accuracy: 0.4755 - val_loss: 2.5212
Epoch 2/20
61/61 672s 11s/step - accuracy: 0.4306 - loss: 2.4078 - val_accuracy: 0.4706 - val_loss: 2.4027
Epoch 3/20
61/61 682s 11s/step - accuracy: 0.4228 - loss: 2.3156 - val_accuracy: 0.4706 - val_loss: 2.3772
Epoch 4/20
61/61 655s 10s/step - accuracy: 0.4404 - loss: 2.2198 - val_accuracy: 0.3873 - val_loss: 2.3352
Epoch 5/20
61/61 721s 11s/step - accuracy: 0.4826 - loss: 2.1134 - val_accuracy: 0.4020 - val_loss: 2.2571
Epoch 6/20
61/61 707s 12s/step - accuracy: 0.4925 - loss: 2.0662 - val_accuracy: 0.4020 - val_loss: 2.2346
Epoch 7/20
61/61 726s 11s/step - accuracy: 0.4728 - loss: 2.0426 - val_accuracy: 0.4314 - val_loss: 2.1837
Epoch 8/20
61/61 748s 11s/step - accuracy: 0.4747 - loss: 1.9235 - val_accuracy: 0.4608 - val_loss: 2.1624
Epoch 9/20
61/61 717s 11s/step - accuracy: 0.4870 - loss: 1.8725 - val_accuracy: 0.4706 - val_loss: 2.0880
Epoch 10/20
61/61 673s 11s/step - accuracy: 0.4956 - loss: 1.8038 - val_accuracy: 0.4755 - val_loss: 2.1089
Epoch 11/20
61/61 650s 11s/step - accuracy: 0.5452 - loss: 1.7287 - val_accuracy: 0.4657 - val_loss: 2.0115
Epoch 12/20
61/61 637s 10s/step - accuracy: 0.4967 - loss: 1.6928 - val_accuracy: 0.3922 - val_loss: 2.2326
Epoch 13/20
61/61 692s 11s/step - accuracy: 0.5119 - loss: 1.6389 - val_accuracy: 0.4608 - val_loss: 2.5010
Epoch 14/20
61/61 687s 11s/step - accuracy: 0.5207 - loss: 1.5713 - val_accuracy: 0.4657 - val_loss: 2.4863
Epoch 15/20
61/61 661s 11s/step - accuracy: 0.5015 - loss: 1.5327 - val_accuracy: 0.4706 - val_loss: 2.5911
Epoch 16/20
61/61 688s 11s/step - accuracy: 0.5042 - loss: 1.5151 - val_accuracy: 0.4706 - val_loss: 1.8426
Epoch 17/20
61/61 651s 11s/step - accuracy: 0.5265 - loss: 1.4607 - val_accuracy: 0.4608 - val_loss: 2.1445
Epoch 18/20
61/61 698s 11s/step - accuracy: 0.5225 - loss: 1.3881 - val_accuracy: 0.4804 - val_loss: 1.8479
Epoch 19/20
61/61 688s 11s/step - accuracy: 0.5239 - loss: 1.3765 - val_accuracy: 0.4363 - val_loss: 1.6929
Epoch 20/20
61/61 665s 11s/step - accuracy: 0.5329 - loss: 1.3374 - val_accuracy: 0.4412 - val_loss: 1.9290

```



ResNet50 Model for Image Classification

Model Architecture

- 50 layers with skip connections to avoid vanishing gradients. Helps train deep networks effectively.
- Convolutional Layers: Starts with a 64-filter convolution (3x3), increasing filters up to 2048.
- Batch Normalization & ReLU: Batch normalization stabilizes training. ReLU activation used for hidden layers.
- Fully Connected Layer: Final layer with softmax for multi-class classification. Weights: Pre-trained on ImageNet, fine-tuned for specific tasks.

Image Preprocessing

- **Data Augmentation:** Random rotations, shifts, zooms, flips
- **Rescaling:** Normalization to [0, 1] for images

Model Training

- **Optimizer:** Adam with learning rate 0.0001
- **Loss Function:** Categorical Crossentropy
- **Callbacks:** EarlyStopping and ModelCheckpoint

Model Analysis

- **Model Analysis :** ResNet50 Model, had training accuracy average % of around 50% and validation accuracy average % of around 50% and during image prediction it was able to detect 1/5 images accurately

Sequential Keras Model for Image Classification

Model Architecture

- A 3-layer CNN with max-pooling, flatten, dense, and dropout layers designed for image classification with 4 output classes.

Image Preprocessing

- Includes normalization, data augmentation, and batch preparation using ImageDataGenerator

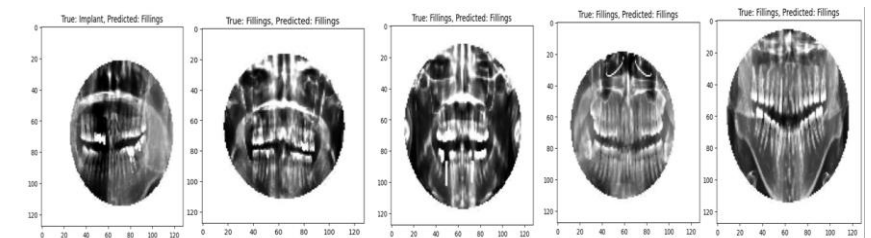
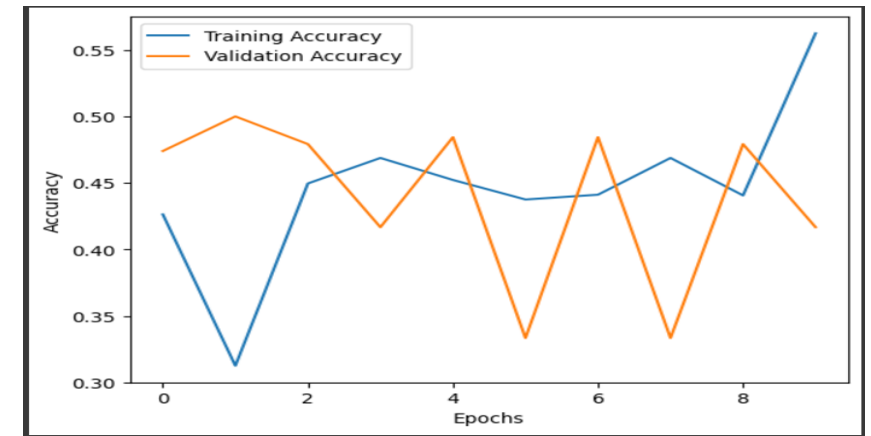
Model Training

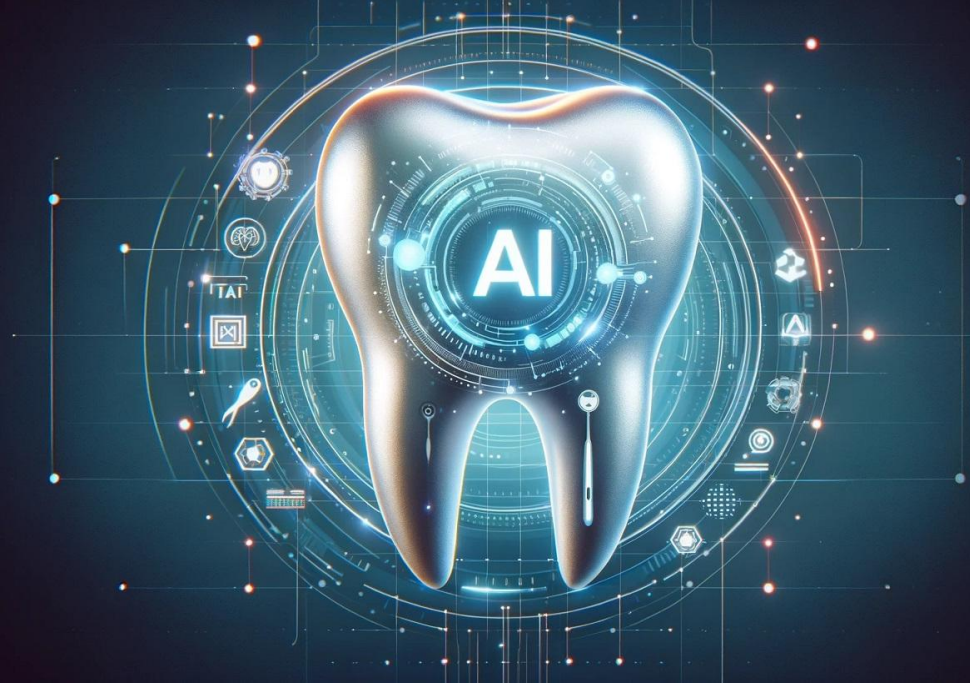
- **Optimizer:** Adam with learning rate 0.0001
- **Loss Function:** Categorical Crossentropy

Model Analysis

- Sequential Model, had training accuracy average % of around 50% and validation accuracy average % of around 50% and during image prediction it was able to detect 4/5 images accurately

```
Epoch 1/10
/usr/local/lib/python3.10/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your
self._warn_if_super_not_called()
60/60 96s 2s/step - accuracy: 0.4081 - loss: 1.3506 - val_accuracy: 0.4740 - val_loss: 1.2225
Epoch 2/10
60/60 1s 3ms/step - accuracy: 0.3125 - loss: 1.3542 - val_accuracy: 0.5000 - val_loss: 1.1772
Epoch 3/10
/usr/local/lib/python3.10/contextlib.py:153: UserWarning: Your input ran out of data; interrupting training. Make sure that y
self.gen.throw(typ, value, traceback)
60/60 88s 1s/step - accuracy: 0.4523 - loss: 1.2390 - val_accuracy: 0.4792 - val_loss: 1.2027
Epoch 4/10
60/60 3s 19ms/step - accuracy: 0.4688 - loss: 1.2750 - val_accuracy: 0.4167 - val_loss: 1.4586
Epoch 5/10
60/60 84s 1s/step - accuracy: 0.4610 - loss: 1.2378 - val_accuracy: 0.4844 - val_loss: 1.2173
Epoch 6/10
60/60 2s 19ms/step - accuracy: 0.4375 - loss: 1.1873 - val_accuracy: 0.3333 - val_loss: 1.3339
Epoch 7/10
60/60 139s 1s/step - accuracy: 0.4589 - loss: 1.2376 - val_accuracy: 0.4844 - val_loss: 1.2196
Epoch 8/10
60/60 3s 22ms/step - accuracy: 0.4688 - loss: 1.2948 - val_accuracy: 0.3333 - val_loss: 1.3252
Epoch 9/10
60/60 86s 1s/step - accuracy: 0.4357 - loss: 1.2542 - val_accuracy: 0.4792 - val_loss: 1.2161
Epoch 10/10
60/60 1s 2ms/step - accuracy: 0.5625 - loss: 1.2122 - val_accuracy: 0.4167 - val_loss: 1.2911
```





Conclusion

- Sequential Model outperformed other 2 models(VGG16 and ResNet50), achieving similar accuracy and was able to predict higher % of the images accurately.
- This project which was aimed to classify dental images into specific categories using advanced deep learning models like VGG16 and ResNet50 and also 1 Sequential model built on Keras.
- Through data augmentation, preprocessing, and fine-tuning of pre-trained networks, the models were trained to capture essential features from the dataset while ensuring robust generalization. Sequential model built on keras both had a training accuracy average validation accuracy average of around 50%.
- During image prediction it was able to predict 4/5 images accurately

